

Princess Sumaya University for Technology

Operating Systems



جامعة
الأميرة سميرة
للتكنولوجيا
Princess Sumaya
University
for Technology

Authors:

Amr Ghuneim	20220056	Software Eng.
Yazan Qattous	20220877	Software Eng.
Amro Kofahi	20210696	Computer Sci.
Anas Ghazal	20221094	Software Eng.
Malek Al-Gourani	20210083	Software Eng.

Supervisor:

Dr. Osama AlHaj Hassan

May 11, 2025

Index

1. Introduction.....	4
2. Background	4
3. Objectives	4
4. Methodology.....	5
4.1 Workload Generation	5
4.2 Hardware Platform.....	5
4.3 Software Environment.....	5
4.4 Experimental Procedure	5
5. Algorithm Summaries	5
5.1 Round Robin (Preemptive).....	5
5.1.1 Advantages & Disadvantages	6
5.1.2 Pseudo Code.....	6
5.2 Priority Scheduling (Non-preemptive)	6
5.2.1 Advantages & Disadvantages	6
5.2.2 Pseudo Code.....	6
5.3 First Come First Served (Non-preemptive)	7
5.3.1 Advantages and Disadvantages	7
5.3.2 Pseudo Code.....	7
5.4 Shortest Job First (Non-preemptive).....	7
5.4.1 Advantages and Disadvantages	7
5.4.2 Pseudo Code.....	8
5.5 Priority Scheduling (Preemptive).....	8
5.5.1 Advantages and Disadvantages	8
5.5.2 Pseudo Code.....	8
6. Current Implementation Status	9
7. Results	9
8. Analysis.....	10
8.1 Shortest-Job-First (Non-Preemptive)	10
8.2 First-Come-First-Served	10
8.3 Priority Scheduling.....	11
8.4 Round Robin.....	11
9. Conclusion	11

10.	Next Steps	11
11.	References.....	12

Figures

Figure 1 - Round Robin Pseudo Code	6
Figure 2 - Priority Scheduling (Non-preemptive) Pseudo Code.....	6
Figure 3 - First Come First Served (Non-preemptive) Pseudo Code.....	7
Figure 4 - Shortest Job First (Non-preemptive) Pseudo Code.....	8
Figure 5 - Priority Scheduling (Preemptive) Pseudo Code.....	8
Figure 6 - Average Turnaround Time by Scheduling Algorithm	9
Figure 7 - Average Waiting Time by Scheduling Algorithm	10

Tables

Table 1 - Average Turnaround Time by Scheduling Algorithm.....	9
Table 2 - Average Waiting Time by Scheduling Algorithm.....	10

Abstract

This report presents a comparative analysis of five representative CPU-scheduling algorithms—Round Robin (preemptive), Priority Scheduling (non-preemptive), Priority Scheduling (preemptive), First-Come-First-Served (FCFS), and Shortest-Job-First (non-preemptive, SJF-NP). These algorithms form the core of process-management strategies in modern operating systems. To approximate real-world workloads, the study executes each scheduler on identical sets of randomly generated processes that differ in burst times, arrival times, and priorities. Performance is quantified with two standard metrics: average turnaround time and average waiting time. By applying controlled, repeatable experiments across varied process counts and service-time distributions, the report highlights how design choices—such as pre-emption rules and priority handling—affect efficiency, fairness, and responsiveness.

1. Introduction

Efficient CPU scheduling is fundamental to responsive, high-throughput computer systems. Because the processor governs every task, its allocation strategy directly shapes turnaround time and overall throughput. In this study we implement and evaluate five classic scheduling algorithms in Python—First-Come First-Served (FCFS), Priority (non-preemptive), Priority (preemptive), Shortest Job First (SJF, non-preemptive) and Round Robin (RR). Although the code is not production-grade and the workloads are randomly generated, running all five under identical conditions reveals each algorithm’s practical strengths and limitations.

2. Background

For six decades operating-system researchers have proposed diverse schedulers to balance fairness, responsiveness and resource utilization. No single technique dominates across all workloads, so comparative studies remain valuable. By measuring common performance metrics on a shared testbed we can isolate how algorithmic policy—not hardware—drives results. [6]

3. Objectives

1. Grasp the operational principles of five representative CPU schedulers.
2. Implement each algorithm in Python and exercise them on the same workload.
3. Quantify average turnaround time and waiting time for every run.
4. Analyze the strengths and weaknesses revealed by the data.

4. Methodology

4.1 Workload Generation

For each of three trials, we built a template list of five processes, assigning each a process ID (P1–P5) and random arrival time, burst time and priority from a uniform 1–10 range. We preserved each template so that every scheduling algorithm ran on the same five-process workload per trial. After executing all algorithms on each template, we aggregated the three turnaround-time and waiting-time results to compute the overall averages.

4.2 Hardware Platform

- Huawei MateBook 14
- Intel Core i5, 11th Generation

4.3 Software Environment

- PyCharm
- No third-party scheduling libraries; all algorithms implemented from first principles.

4.4 Experimental Procedure

- Each scheduling function (FCFS, Priority preemptive and non-preemptive, SJF non-preemptive, Round Robin) was executed on the same workload.
- The process list was deep-cloned before each run to prevent state carry-over between algorithms.
- Every experiment was repeated multiple times with newly generated workloads; average turnaround time and average waiting time were recorded for every run.
- Results were aggregated and discussed in an online group meeting (Discord) to ensure consensus on interpretation and next steps.

5. Algorithm Summaries

5.1 Round Robin (Preemptive)

Allocates a fixed **time quantum** to each ready process in cyclic order. A job that exhausts its slice without finishing is re-queued. This ensures fairness but incurs context-switch overhead; if the quantum is too small throughput and waiting time suffer. [1]

Example:

Two processes (P1 = 10 ms, P2 = 10 ms) and quantum = 3 ms: the CPU executes

P1(3) → P2(3) → P1(3) → P2(3) → P1(3) → P2(3) → P1(1) → P2(1).

5.1.1 Advantages & Disadvantages

- **Advantage:** Ensures no process starves—every job gets CPU time at regular intervals.
- **Disadvantage:** Frequent context switches add overhead, especially if the quantum is too small.

5.1.2 Pseudo Code

```
1. Initialize queue with all processes
2. While processes remain in the queue:
  a. Select the next process in the queue
  b. Execute the process for a time slice (quantum) or until completion
  c. If process is not finished, update remaining burst time and re-enqueue
  d. If process finishes, calculate wait time and turnaround time
3. Output average wait time and turnaround time
```

Figure 1 - Round Robin Pseudo Code

5.2 Priority Scheduling (Non-preemptive)

Selects the highest-priority process in the ready queue; ties break on arrival time. The chosen job runs to completion. While simple, this policy risks starvation of low-priority jobs; aging techniques can mitigate the problem. [6]

Example:

P1(priority 1), P2(2), P3(3) $\Rightarrow$ execution order: P1 $\rightarrow$ P2 $\rightarrow$ P3, independent of arrival order.

5.2.1 Advantages & Disadvantages

- **Advantage:** Low implementation complexity and predictable run-to-completion behavior for high-priority tasks.
- **Disadvantage:** Lower-priority processes may wait indefinitely (starvation) if high-priority arrivals continue.

5.2.2 Pseudo Code

```
1. Sort processes by arrival time, then by priority
2. While unprocessed processes remain:
  a. Select highest priority (lowest value)
  b. Execute to completion
  c. Calculate wait time = start time - arrival time
  d. Calculate turnaround time = finish time - arrival time
  e. Add wait and turnaround to totals
3. Output average wait time and turnaround time
```

Figure 2 - Priority Scheduling (Non-preemptive) Pseudo Code

5.3 First Come First Served (Non-preemptive)

Executes jobs strictly by arrival time. FCFS is easy to implement and perceived as fair, but long jobs delay all subsequent tasks—known as the convoy effect—leading to poor average waiting time in mixed-length workloads. [4]

Example:

Arrival sequence P1, P2, P3 → execution remains P1 → P2 → P3 even if P1 is much longer.

5.3.1 Advantages and Disadvantages

- **Advantage:** Minimal context-switch overhead and simple queue management.
- **Disadvantage:** The convoy effect can dramatically increase waiting time for short jobs arriving behind long ones.

5.3.2 Pseudo Code

```
1. Sort processes by arrival time
2. While unprocessed processes remain:
  a. Select the next process (in arrival order)
  b. Execute to completion
  c. Calculate wait time = start time - arrival time
  d. Calculate turnaround time = finish time - arrival time
  e. Add wait and turnaround to totals
3. Output average wait time and turnaround time
```

Figure 3 - First Come First Served (Non-preemptive) Pseudo Code

5.4 Shortest Job First (Non-preemptive)

Always selects the job with the smallest next CPU burst among arrived tasks, minimizing mean waiting time when burst lengths are known. Performance degrades if burst estimates are inaccurate or highly variable. [5]

Example:

Burst times P1 = 6 ms, P2 = 2 ms, P3 = 8 ms ⇒ order: P2 → P1 → P3.

5.4.1 Advantages and Disadvantages

- **Advantage:** Provably minimal average waiting time under perfect burst knowledge.
- **Disadvantage:** Requires advance knowledge or accurate prediction of burst lengths; older jobs may starve if many short jobs arrive.

5.4.2 Pseudo Code

```
1. Sort processes by arrival time
2. While unprocessed processes remain:
  a. Select the shortest job (smallest burst time) that has arrived
  b. Execute to completion
  c. Calculate wait time = start time - arrival time
  d. Calculate turnaround time = finish time - arrival time
  e. Add wait and turnaround to totals
3. Output average wait time and turnaround time
```

Figure 4 - Shortest Job First (Non-preemptive) Pseudo Code

5.5 Priority Scheduling (Preemptive)

When a process with a higher priority (numerically lower value) arrives, the running task is immediately preempted and the CPU is reassigned. This improves responsiveness for critical jobs but can starve low-priority work unless an aging mechanism gradually raises their priority. [6]

Example:

Suppose P1 (priority 3) is running and P2 (priority 1) arrives at time 4 ms; P1 is paused, P2 runs to completion (or until an even higher-priority job appears), after which P1 resumes.

5.5.1 Advantages and Disadvantages

- **Advantage:** High-priority tasks get served immediately, reducing critical-job latency.
- **Disadvantage:** Preemption overhead and risk of starvation for lower-priority tasks without aging.

5.5.2 Pseudo Code

```
1. Sort processes by arrival time, then by priority
2. While unprocessed processes remain:
  a. Select highest priority (lowest value) process that is ready to execute
  b. Execute for 1 unit of time or until completion
  c. Update remaining burst time
  d. Recalculate wait time and turnaround time after each execution
  e. Preempt if a higher priority process arrives
3. Output average wait time and turnaround time
```

Figure 5 - Priority Scheduling (Preemptive) Pseudo Code

6. Current Implementation Status

- **Code base:** Five standalone Python functions, each returning average turnaround and waiting times for a supplied list of Process objects.
- **Validation:** Algorithms exercised on identical random workloads to ensure fair comparison; each run repeated to obtain stable averages.
- **Division of labor:**
 - **Amr Ghuneim:** Round Robin - Preemptive
 - **Yazan:** Priority Scheduling - Non-Preemptive
 - **Malek:** First-Come-First-Served - Non-Preemptive
 - **Amro Kofahi:** Shortest Job First - Non-Preemptive
 - **Anas:** Priority Scheduling – Preemptive

7. Results

Table 1 - Average Turnaround Time by Scheduling Algorithm

Algorithm	TAT 1	TAT 2	TAT 3	Avg TAT
FCFS	18.80	12.40	15.80	15.67
Prio-NP	24.80	18.80	12.60	18.73
Prio-P	19.60	18.00	12.40	16.67
RR	24.00	13.60	13.00	16.87
SJF-NP	16.20	10.60	12.60	13.13

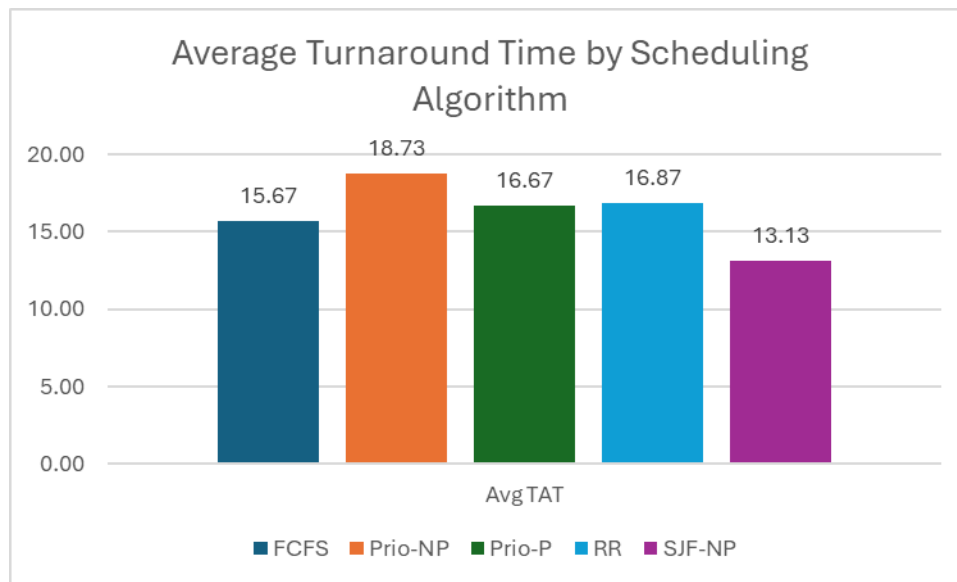


Figure 6 - Average Turnaround Time by Scheduling Algorithm

Table 2 - Average Waiting Time by Scheduling Algorithm

Algorithm	WT 1	WT 2	WT 3	Avg WT
FCFS	11.60	7.20	9.80	9.53
Prio-NP	17.60	13.60	6.60	12.60
Prio-P	12.40	12.80	6.40	10.53
RR	19.00	10.20	10.00	13.07
SJF-NP	9.00	5.40	6.60	7.00

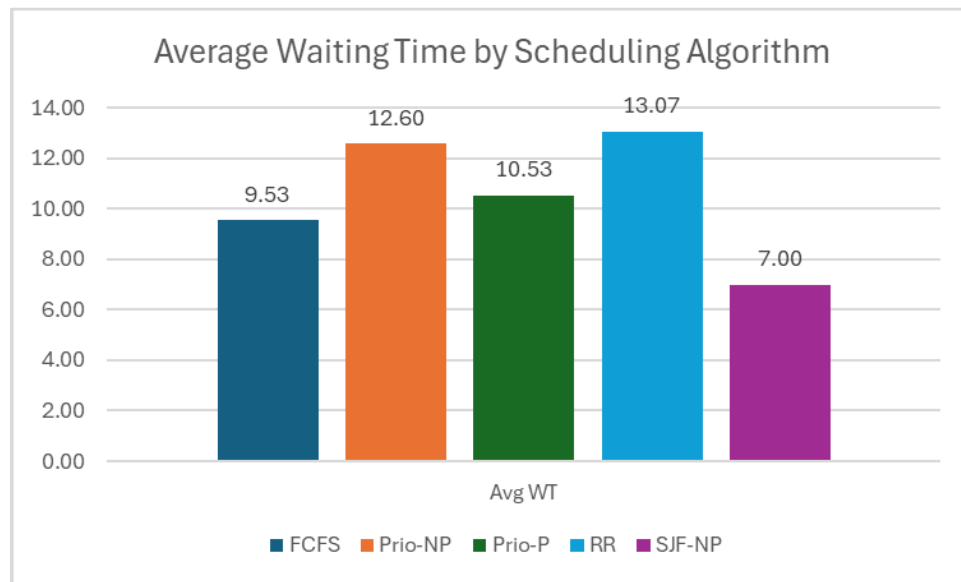


Figure 7 - Average Waiting Time by Scheduling Algorithm

8. Analysis

8.1 Shortest-Job-First (Non-Preemptive)

- **Best overall:** Lowest Avg TAT (13.13 ms) and lowest Avg WT (7.00 ms).
- **Consistency:** TATs range 10.60–16.20 ms; WTs 5.40–9.00 ms
- **Insight:** By always selecting the smallest burst, SJF minimizes both metrics when burst lengths are known.

8.2 First-Come-First-Served

- **Moderate performance:** Avg TAT = 15.67 ms, Avg WT = 9.53 ms.
- **Variability:** TAT swings 12.40–18.80 ms; WT swings 7.20–11.60 ms.
- **Observation:** Simple fairness comes at the cost of convoy-effect delays when long jobs enter early.

8.3 Priority Scheduling

- **Non-Preemptive vs. Preemptive:**
 - **Non-preemptive** Avg TAT = 18.73 ms, Avg WT = 12.60 ms (worst performer).
 - **Preemptive:** Avg TAT = 16.67 ms, Avg WT = 10.53 ms (≈ 2 ms improvement over its non-preemptive counterpart).
- **Interpretation:** Allowing preemption for higher-priority arrivals significantly reduces both turnaround and waiting times, mitigating starvation.

8.4 Round Robin

- **Fair but costly:** Avg TAT = 16.87 ms, Avg WT = 13.07 ms (highest waiting time).
- **Switch-overhead effect:** Frequent context switches (quantum = 3 ms) increase waiting time, especially for I/O-bound or short processes.

9. Conclusion

1. **SJF-NP** is the most efficient when burst times are known in advance, achieving the lowest average turnaround and waiting times.
2. **Priority-Preemptive** offers a good compromise for mixed-priority workloads, improving significantly over its non-preemptive form by reacting to urgent tasks.
3. **Priority Non-Preemptive** is straightforward to implement and predictable, but can lead to higher average waiting times and starvation of low-priority processes.
4. **FCFS** remains simple but suffers from high variability and the convoy effect.
5. **Round Robin**, while fair, incurs the highest waiting times due to context-switch overhead; tuning the quantum size could balance responsiveness and efficiency.

For systems where burst estimates are unavailable or fairness across jobs is paramount, **Priority-Preemptive** (with aging) or a well-tuned **Round Robin** are recommended. When task lengths can be reasonably predicted, **SJF-NP** delivers optimal average performance.

10. Next Steps

1. Extend the workload generator to include burst-time distributions that mimic real-world traces.
2. Add preemptive variants (e.g., SRTF) for deeper comparison.
3. Incorporate context-switch overhead explicitly into timing metrics.
4. Visualize Gantt charts to illustrate scheduling behavior qualitatively.

11. References

1. Preemptive and Non-Preemptive Scheduling
 - <https://www.geeksforgeeks.org/difference-between-preemptive-and-non-preemptive-cpu-scheduling-algorithms/>
2. Round Robin (RR) CPU Scheduling Algorithm in OS with Example
 - <https://www.youtube.com/watch?v=-jFGYDfWkXI&t=6s>
3. Scheduling Algorithms – First Come First Served (FCFS)
 - <https://www.youtube.com/watch?v=7DoP1L9nAAs>
4. Preemptive vs Non-Preemptive Scheduling
 - <https://www.youtube.com/watch?v=4DhFmL-6SDA>
5. Shortest Job First – Preemptive Scheduling with Example (SJF)
 - <https://prepinsta.com/operating-systems/shortest-job-first-scheduling-preemptive-example/>
6. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons. See Chapter 6 “CPU Scheduling.” Page 332
 - <https://archive.org/details/operatingsystemconcepts10th/page/n331/mode/2up>